1. Dar formato

Para dar formato a los formularios, puede utilizar las propiedades CSS aplicables al texto, así como las propiedades de los contenedores (ancho, contorno, color de fondo...). No existe ninguna propiedad ni ningún módulo específicamente dedicado a los formularios.

2. Redimensionar un campo

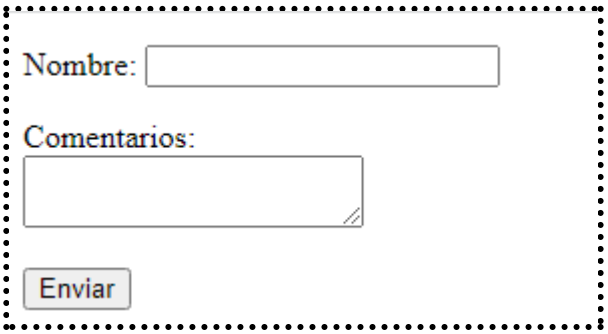
La propiedad `resize` puede usarse, por ejemplo, en un campo de formulario cuando el diseñador desee que el usuario pueda modificar el tamaño de dicho campo.

Propiedad: resize

Valor: none | both | horizontal | vertical

Los valores que se pueden utilizar:

- `none`: sin posibilidad de redimensionamiento.
- `both`: redimensionable horizontal y verticalmente.
- `horizontal`: redimensionable horizontalmente.
- `vertical`: redimensionable verticalmente.



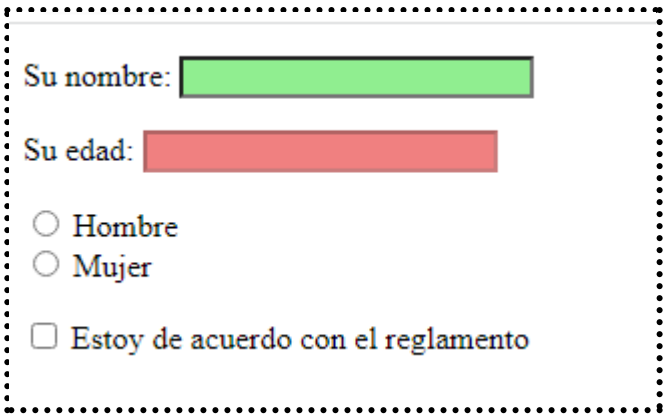

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
   <title>Redimensionar un campo</title>
   <meta charset="UTF-8" />
   <style>
      /* #comentario {
         width: 200px;
         height: 90px;
         border: 1px solid #333;
         background-color: lightyellow;
         resize: both;
      }    */
   </style>
</head>
<body>
<form id="test" method="#" action="#">
   <p>
      <label for="nombre">Nombre: </label>
      <input type="text" id="nombre" />
   </p>
   <p>
      <label for="comentario">Comentarios: </label>
      <br />
      <textarea name="comentario" id="comentario"></textarea>
   </p>
   <p>
      <input type="submit" name="envio" id="envio" value="Enviar" />
   </p>
</form>
</body>
</html>
```

3. Pseudoclases para formularios

Las CSS3 proponen una serie de pseudoclases muy útiles para los objetos de formulario; permiten resaltar los objetos activos, desactivados y marcados en un formulario. De este modo, el usuario puede saber qué ha hecho él, qué está activo y qué inactivo.

Esta nuevas pseudoclases son: `:enabled`, `:disabled` y `:checked`.



El primer selector se dirige a la etiqueta (`label`) de la casilla de verificación (`#acuerdo`) para saber si está marcada (`checked`). Lo que queremos resaltar es, efectivamente, la etiqueta de la casilla de verificación, y no la casilla de verificación propiamente dicha; de ahí el uso de un selector adyacente.

El segundo selector se dirige al botón de opción marcado, también en este caso para detectar que lo está.

Los otros dos estilos simplemente sirven para atribuir un fondo de color según el estado del campo.

```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>Pseudoclases para los formularios</title>
   <meta charset="UTF-8" />
   <style>
      /* #acuerdo:checked+label {
         background-color: gold;
         font-weight: bold;
      }    */
      /* #hombre:checked+label, #mujer:checked+label {
         font-style: italic;
      }    */
      /* :enabled {
         background-color: lightgreen;
      }    */
      /* :disabled {
         background-color: lightcoral;
      }    */
   </style>
```

```
</head>
<body>
<form id="form1" method="#" action="#">
<p>
   <label for="nombre">Su nombre: </label>
    <input type="text" id="nombre" />
</p>
<p>
    <label for="edad">Su edad: </label>
    <input type="text" id="edad" disabled="disabled"/>
</p>
<p>
   <input type="radio" name="sexo" id="hombre" value="hombre" />
   <label for="hombre">Hombre</label><br/>
   <input type="radio" name="sexo" id="mujer" value="mujer" />
   <label for="hombre">Mujer</label>
</p>
<p>
    <input type="checkbox" id="acuerdo" />
    <label for="acuerdo">Estoy de acuerdo con el reglamento</label>
</p>
</form>
</body>
</html>
```

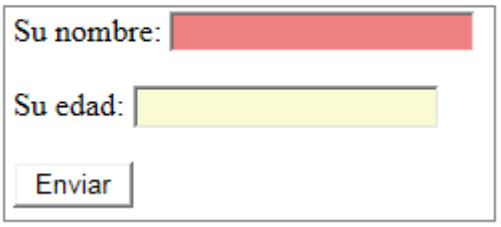
4. Los campos requeridos y los opcionales

Si lo desea, puede resaltar los campos que sean obligatorios u opcionales con las pseudoclases `:required` y `:optional`.

Los campos que son obligatorios, es decir, aquellos en los que se exige introducir o marcar algo, deben disponer del atributo booleano `required`. Los que no sean obligatorios serán opcionales de forma predeterminada, sin que resulte necesario especificarlo.

He aquí un ejemplo (**06_05.html**):

Esto es lo que se obtiene:




```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>Los campos requeridos y los opcionales</title>
   <meta charset="UTF-8" />
   <style>
      /* input:required {
         background-color: lightcoral;
      }    */
      /* input:optional {
         background-color: lightgoldenrodyellow;
      }    */
      /* input#envio {
         background: none;
      }    */
   </style>
</head>
<body>
<form id="form1" method="#" action="#">
  <p>
    <label for="nombre">Su nombre: </label>
    <input type="text" name="nombre" id="nombre" required />
  </p>
  <p>
    <label for="edad">Su edad: </label>
    <input type="text" name="edad" id="edad" />
  </p>
  <p>
     <input type="submit" id="envio" value="Enviar" />
  </p>
</form>
</body>
</html>
```

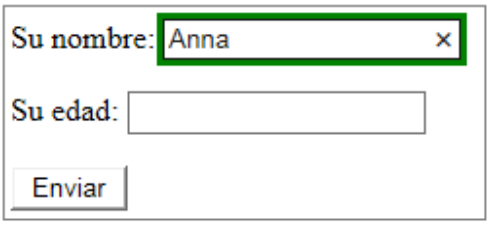
5. Dar formato al «foco»

La pseudoclase `:focus` sirve para resaltar el campo que se está usando mediante la adición de un contorno suplementario que proporciona la propiedad `outline`. Las CSS3 añaden una nueva

propiedad: `outline-offset`, que define la distancia entre el límite de la caja y el contorno de resalte

He aquí un ejemplo muy sencillo (**06_06.html**):

He aquí lo que aparece cuando el usuario hace clic en un campo:



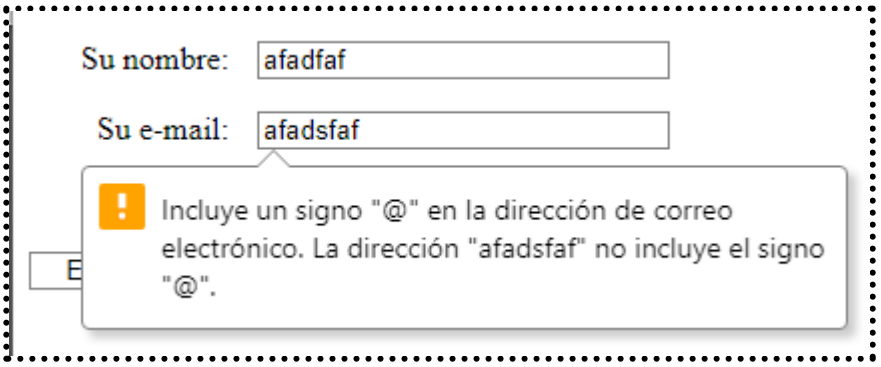

```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>El foco</title>
   <meta charset="UTF-8" />
   <style>
      /* input:focus {
         outline: solid 3px green;
         outline-offset: 2px;
      }    */
      /* input#envio {
         background: none;
      }    */
   </style>
</head>
<body>
<form id="form1" method="#" action="#">
  <p>
    <label for="nombre">Su nombre: </label>
    <input type="text" name="nombre" id="nombre" required />
  </p>
  <p>
    <label for="edad">Su edad: </label>
    <input type="text" name="edad" id="edad" />
  </p>
  <p>
    <input type="submit" id="envio" value="Enviar" />
  </p>
```

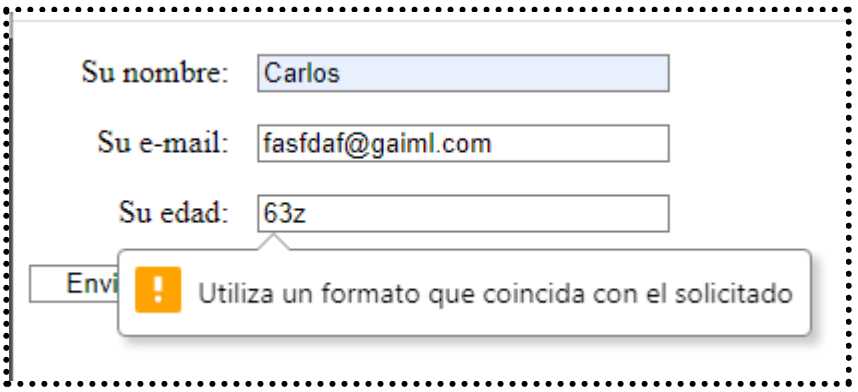
```
</form>
</body>
</html>
```

6. Validar la información introducida

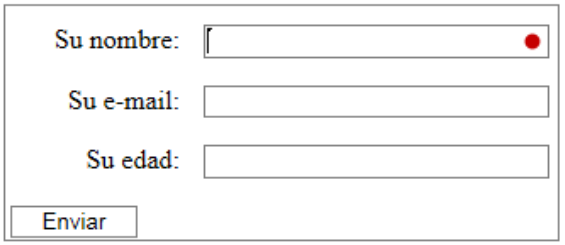
Se trata de resaltar la validación de los campos en un formulario. En este ejemplo (**06_07.html**), los dos primeros campos son obligatorios debido al uso del atributo `required`. El tercer campo incorpora un patrón que deben seguir los datos que se introduzcan (`pattern`). En estos tres campos se mostrará un círculo rojo cuando la información introducida no sea válida, y un círculo verde cuando sí lo sea.

Veamos el código de este ejemplo:



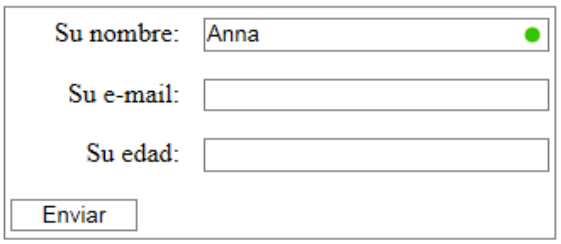


Esto es lo que se muestra cuando el usuario hace clic en el primer campo, **Su nombre**:



El campo no es válido, ya que no se ha introducido nada y es obligatorio rellenar este campo.

Veamos lo que aparece cuando el usuario escribe algo en el primer campo, **Su nombre**:




```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>Validación de un formulario</title>
   <meta charset="UTF-8" />
   <style>
      /* input {
         border: solid 1px gray;
         width: 200px;
      }    */
      /* label, input{
         display: inline-block;
      }    */
      /* label {
         width: 100px;
         text-align: right;
         margin-right: 10px;
      }    */
      /* input[type=submit]{
         width: 75px;
         background: none;
      }    */
      /* input:focus:invalid {
         background: url(rojo.png) no-repeat 98% center;
      }    */
      /* input:focus:valid {
         background: url(verde.png) no-repeat 98% center;
```

```
        }    */
      </style>
   </head>
   <body>
   <form id="inscription" method="#" action="#">
      <p>
         <label for="nombre">Su nombre: </label>
         <input type="text" id="nombre" required />
      </p>
      <p>
         <label for="email">Su e-mail: </label>
         <input type="email" id="email" required />
      </p>
      <p>
         <label for="edad">Su edad: </label>
         <input type="text" id="edad" pattern="[0-9]]{2}" />
      </p>
      <p>
         <input type="submit" id="envio" value="Enviar" />
      </p>
   </form>
   </body>
   </html>
```

Los contornos dinámicos

El contorno dinámico sirve para resaltar un elemento de la página, principalmente en el momento en que el usuario realiza una acción. El ejemplo más común es su utilización en los campos de un formulario.

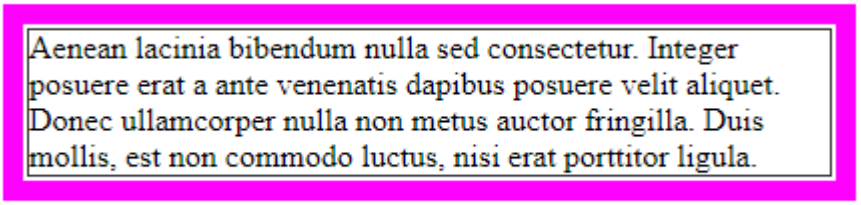
La propiedad `outline` es la sintaxis corta de estas propiedades individuales: `outline-color`, `outline-style` y `outline-width`.

Utilizaremos también la propiedad `outline-offset`, que sirve para definir la distancia entre el elemento y su contorno dinámico.

Veamos un ejemplo (**07_01.html**) muy sencillo de contorno dinámico en un párrafo en el que el usuario ha hecho clic

```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>Párrafo resaltado</title>
   <meta charset="UTF-8" />
   <style>
      /* body {
```

```
            margin: 30px;
        }    */
        /* .cuadro {
            width: 400px;
            border: 1px solid #333;
        }    */
        /* .cuadro:active {
            outline: fuchsia solid 10px;
            outline-offset: 2px;
        }    */
    </style>
</head>
<body>
    <p class="cuadro">Aenean lacinia bibendum nulla sed
consectetur. Integer posuere erat a ante venenatis dapibus
posuere velit aliquet. Donec ullamcorper nulla non metus
auctor fringilla. Duis mollis, est non commodo luctus,
nisi erat porttitor ligula.</p>
</body>
</html>
```



Los cursores

La propiedad `cursor` permite cambiar la apariencia del cursor en un contexto determinado de la página.

He aquí los valores que pueden usarse:

Cursores generales:

- `url`: indica la ruta de acceso al archivo de imagen que reemplaza al cursor.
- `auto`: el navegador escoge la apariencia del cursor.

- `default`: cursor por defecto de la plataforma utilizada (con frecuencia es una flecha).
- `none`: sin cursor.

Cursores de vínculo y estado:

- `content-menu`: para mostrar el cursor por encima de un elemento.
- `help`: el cursor puede tomar el aspecto de un signo de interrogación o un globo para indicar que el elemento señalado dispone de ayuda.
- `pointer`: el cursor toma la apariencia de un dedo extendido.
- `progress`: el cursor toma la apariencia de una rueda o de una flecha con un reloj para indicar que se está llevando a cabo un proceso.
- `wait`: el cursor toma la apariencia de un reloj (que puede ser de arena) para indicar que hay que esperar.

Cursores para seleccionar:

- `cell`: para seleccionar celdas.
- `crosshair`: cursor en forma de cruz.
- `text`: indica que puede seleccionarse texto. El cursor toma la apariencia de una I.
- `vertical-text`: indica que puede seleccionarse un texto vertical. El cursor toma la apariencia de una I horizontal.

Cursores para hacer clic y arrastrar-soltar:

- `alias`: indica el alias de un elemento. Usualmente se trata de una flecha con una curva.
- `copy`: indica que se ha copiado un elemento. Usualmente se trata de una flecha con un signo +.
- `move`: indica que un objeto puede moverse.

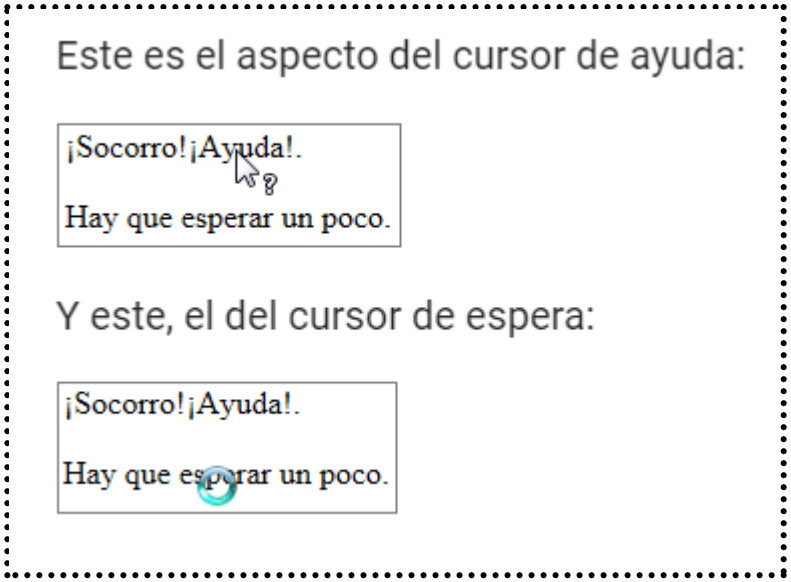
- `no-drop`: indica que un elemento no puede soltarse. Usualmente se trata de una flecha o una mano con un signo de prohibición.
- `not-allowed`: indica una acción no autorizada. Usualmente se trata de un signo de prohibición.
- `grab`: indica que algo puede «agarrarse». Se representa con un dorso de mano abierta.
- `grabbing`: indica que se tiene algo «agarrado» y que se está moviendo. Se representa con un puño.

Cursores de cambio de tamaño y desplazamiento:

- `e-resize`, `n-resize`, `ne-resize`, `nw-resize`, `s-resize`, `se-resize`, `sw-resize`, `w-resize`: indica que el borde de un objeto puede desplazarse. Los valores son coordenadas «geográficas»: `ne` para nordeste, `sw` para sudoeste...
- `ew-resize`, `ns-resize`, `nesw-resize`, `nwse-resize`: cursores bidireccionales.
- `col-resize`: indica que puede cambiarse el tamaño de una columna.
- `row-resize`: indica puede cambiarse el tamaño de una fila.
- `all-resize`: indica que un objeto puede redimensionarse en todas las direcciones.

Cursores de zoom:

- `zoom-in`, `zoom-out`: indica la posibilidad de aplicar un zoom. Usualmente se trata de una lupa con los signos + y -.




```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>Cursores</title>
   <meta charset="UTF-8" />
   <style>
      /* .ayuda {
         cursor: help;
      }    */
      /* .espera {
         cursor: pointer;
      }    */
   </style>
</head>
<body>
   <p>¡Socorro!<span class="ayuda"> ¡Ayuda!</span>.</p>
   <p>Hay que <span class="espera">esperar</span> un poco.</p>
</body>
</html>
```